

Assignment 3 Report

REN Kaiyan 21270191

May 9, 2026

1 MDP Value Iteration for the Wumpus World

1.1 Problem Description

We solve the 4×4 Wumpus World problem using MDP Value Iteration. The grid layout is as follows (states numbered 0–15, row-major order):

Start (0)	(1)	(2)	Gold (3)
(4)	(5)	(6)	(7)
(8)	(9)	Pit (10)	(11)
(12)	Wumpus (13)	Pit (14)	(15)

The MDP is defined with the following parameters:

- **States:** $S = \{0, 1, 2, \dots, 15\}$
- **Actions:** $A = \{\text{UP, DOWN, LEFT, RIGHT}\}$
- **Transition model:** 0.8 probability of moving in the intended direction, 0.1 probability of moving in each perpendicular direction. If blocked by a wall, the agent stays in the same state.
- **Rewards:** $R(\text{Gold}) = +10$, $R(\text{Wumpus}) = -10$, $R(\text{Pit}) = -5$, $R(\text{other}) = -0.4$

1.2 Results

We run Value Iteration with convergence threshold $\eta = 0.1$ and maximum iterations = 10000 for three discount factors: $\gamma \in \{0.3, 0.6, 0.9\}$.

1.2.1 Setting I: $\gamma = 0.3, \eta = 0.1$

Table 1: Utilities and Policy for $\gamma = 0.3$

0.15 $\rightarrow$	2.39 $\rightarrow$	11.37 $\rightarrow$	12.71 $\rightarrow$
-0.37 $\rightarrow$	0.26 $\rightarrow$	2.66 $\uparrow$	11.37 $\uparrow$
-0.54 $\uparrow$	-0.82 $\uparrow$	0.25 $\uparrow$	1.91 $\uparrow$
-0.57 $\leftarrow$	-1.13 $\uparrow$	-1.48 $\rightarrow$	-0.48 $\uparrow$

1.2.2 Setting II: $\gamma = 0.6, \eta = 0.1$

Table 2: Utilities and Policy for $\gamma = 0.6$

4.91 $\rightarrow$	10.24 $\rightarrow$	20.30 $\rightarrow$	22.01 $\rightarrow$
2.53 $\rightarrow$	5.49 $\rightarrow$	10.84 $\uparrow$	20.30 $\uparrow$
0.95 $\uparrow$	2.11 $\uparrow$	5.46 $\uparrow$	9.74 $\uparrow$
-0.79 $\leftarrow$	0.11 $\uparrow$	0.97 $\rightarrow$	4.07 $\uparrow$

1.2.3 Setting III: $\gamma = 0.9, \eta = 0.1$

Table 3: Utilities and Policy for $\gamma = 0.9$

62.52 $\rightarrow$	72.72 $\rightarrow$	84.56 $\rightarrow$	86.77 $\rightarrow$
55.76 $\rightarrow$	64.17 $\rightarrow$	73.78 $\uparrow$	84.56 $\uparrow$
49.07 $\uparrow$	55.45 $\uparrow$	64.13 $\uparrow$	72.21 $\uparrow$
41.94 $\uparrow$	47.56 $\uparrow$	53.36 $\rightarrow$	61.37 $\uparrow$

1.3 Analysis

Several observations can be made from the results:

Effect of γ : As the discount factor increases from 0.3 to 0.9, the utility values of all states increase significantly. This is because a higher γ means the agent values future rewards more heavily. With $\gamma = 0.9$, the Gold reward at state 3 propagates much further across the grid, resulting in high utility even at distant states.

Policy structure: Across all three settings, the optimal policy consistently directs the agent toward the Gold (state 3) while avoiding the Wumpus (state 13) and the Pits (states 10 and 14). In the top two rows, the policy moves right and then up toward the Gold. In the bottom rows, the policy moves upward to escape the dangerous region.

Dangerous states: States adjacent to the Wumpus and Pits have notably lower utilities. In particular, state 12 has a LEFT policy for low γ values, reflecting the agent's desire to avoid accidentally moving into the Wumpus at state 13. However, with $\gamma = 0.9$, state 12's policy switches to UP because the long-term benefit of reaching the Gold outweighs the local risk, and moving up is the safest escape route.

2 Alpha-Beta Pruning on a Game Tree

2.1 Problem Description

We are given the following game tree with MAX at the root. The tree has three levels: MAX $\rightarrow$ MIN $\rightarrow$ MAX $\rightarrow$ terminal nodes. The terminal (leaf) values from left to right are:

3, 5, 6, 9, 1, 2, 0, 7

2.2 Game Tree Structure

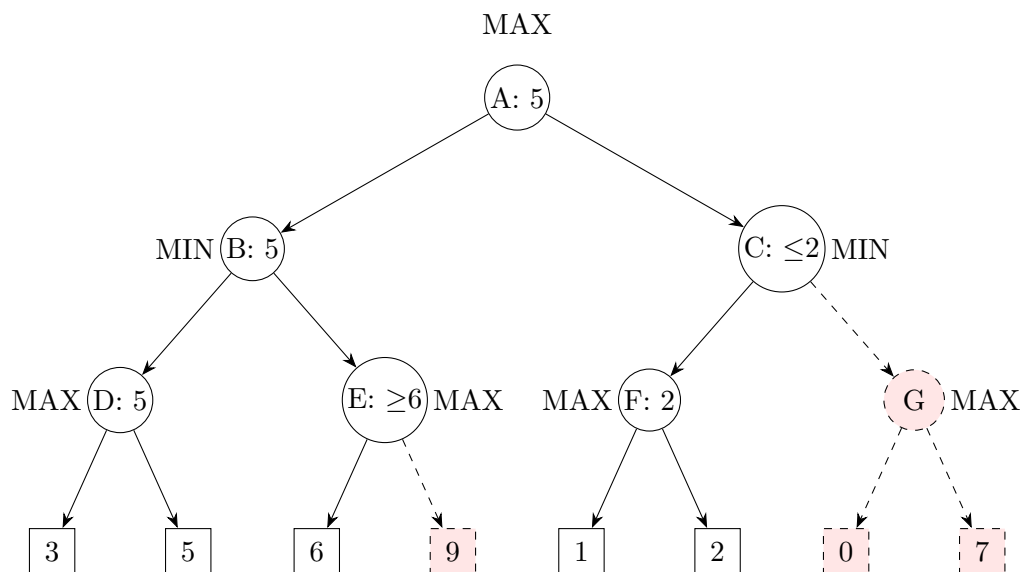


Figure 1: Game tree with alpha-beta pruning. Dashed nodes/edges indicate pruned branches.

2.3 Minimax Values (Without Pruning)

For reference, the standard minimax values computed without pruning are:

$$\begin{aligned} D &= \max(3, 5) = 5, & E &= \max(6, 9) = 9 \\ F &= \max(1, 2) = 2, & G &= \max(0, 7) = 7 \\ B &= \min(5, 9) = 5, & C &= \min(2, 7) = 2 \\ A &= \max(5, 2) = 5 \end{aligned}$$

2.4 Alpha-Beta Pruning Trace

We now trace the alpha-beta algorithm (left-to-right, depth-first search). Each node inherits α and β bounds from its parent.

Step 1: Begin at node A (MAX). Initialize $\alpha = -\infty$, $\beta = +\infty$.

Step 2: Move to node B (MIN). Inherit $\alpha = -\infty$, $\beta = +\infty$.

Step 3: Move to node D (MAX). Inherit $\alpha = -\infty$, $\beta = +\infty$.

Step 4: Evaluate leaf value 3. At D (MAX): update $\alpha = \max(-\infty, 3) = 3$. Check: $\alpha(3) < \beta(+\infty)$, no pruning.

Step 5: Evaluate leaf value 5. At D (MAX): update $\alpha = \max(3, 5) = 5$. Check: $\alpha(5) < \beta(+\infty)$, no pruning. Node D returns value **5**.

Step 6: Back at node B (MIN). Update: $\beta = \min(+\infty, 5) = 5$. Check: $\alpha(-\infty) < \beta(5)$, no pruning.

Step 7: Move to node E (MAX). Inherit $\alpha = -\infty, \beta = 5$ (from B).

Step 8: Evaluate leaf value 6. At E (MAX): update $\alpha = \max(-\infty, 6) = 6$. Check: $\alpha(6) \geq \beta(5)$. **PRUNE!** The remaining child of E (leaf value 9) is not explored.

Reason: E is a MAX node and its value is already ≥ 6 . Since B is a MIN node with current best value 5, B will never choose E (because $6 > 5$). Therefore, further exploration of E is unnecessary.

Step 9: Back at node B (MIN). The value returned by E is 6. Since $\min(5, 6) = 5$, node B's value remains **5**. Node B returns value **5**.

Step 10: Back at node A (MAX). Update: $\alpha = \max(-\infty, 5) = 5$. Check: $\alpha(5) < \beta(+\infty)$, no pruning.

Step 11: Move to node C (MIN). Inherit $\alpha = 5, \beta = +\infty$ (from A).

Step 12: Move to node F (MAX). Inherit $\alpha = 5, \beta = +\infty$.

Step 13: Evaluate leaf value 1. At F (MAX): $\alpha = \max(5, 1) = 5$ (unchanged). Check: $\alpha(5) < \beta(+\infty)$, no pruning.

Step 14: Evaluate leaf value 2. At F (MAX): $\alpha = \max(5, 2) = 5$ (unchanged). Check: $\alpha(5) < \beta(+\infty)$, no pruning. Node F returns value $\max(1, 2) = \mathbf{2}$.

Step 15: Back at node C (MIN). Update: $\beta = \min(+\infty, 2) = 2$. Check: $\alpha(5) \geq \beta(2)$. **PRUNE!** The entire subtree rooted at G (leaves 0 and 7) is not explored.

Reason: C is a MIN node with current value ≤ 2 . Since A is a MAX node with $\alpha = 5$, A will never choose C (because $2 < 5$). Therefore, further exploration of C's children is unnecessary.

Step 16: Back at node A (MAX). The value of A is $\max(5, 2) = \mathbf{5}$.

2.5 Summary of Pruning

Table 4: Summary of Alpha-Beta Pruning

Pruning Event	Location	Nodes Pruned
Pruning 1	Node E (MAX), $\alpha \geq \beta$ ($6 \geq 5$)	Leaf with value 9
Pruning 2	Node C (MIN), $\alpha \geq \beta$ ($5 \geq 2$)	Node G (leaves 0, 7)

In total, 3 leaf nodes out of 8 were pruned (leaves with values 9, 0, and 7), meaning only 5 leaf evaluations were needed. The final minimax value at the root is **5**, which is the same as the full minimax result, confirming that alpha-beta pruning does not affect the optimal decision.

2.6 Optimal Decision

The optimal move for the MAX player at the root is to choose the left branch (toward node B), which guarantees a utility of 5 regardless of the MIN player's strategy.